

Clase 3 – Fundamentos del Web Moderno

IIC2182 – Interfaces y Experiencia de Usuario



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

Primer Semestre 2026

¿Qué aprenderemos hoy?

 HTML: estructura de la web

 CSS: presentacion visual

 Flexbox y Grid: layout moderno

 Tailwind CSS: utility-first

Parte 1: HTML

La estructura y el significado del contenido web

¿Que es HTML?

HyperText Markup Language



Lenguaje de marcado

HTML **no es un lenguaje de programacion**. Es un lenguaje de marcado que define el significado y la estructura del contenido web.

Usa **etiquetas (tags)** para describir que es cada elemento.

</> Ejemplo basico

```
<p class="mi-planta">
  Mi planta es muy verde
</p>


```

Cada tag describe contenido + metadata



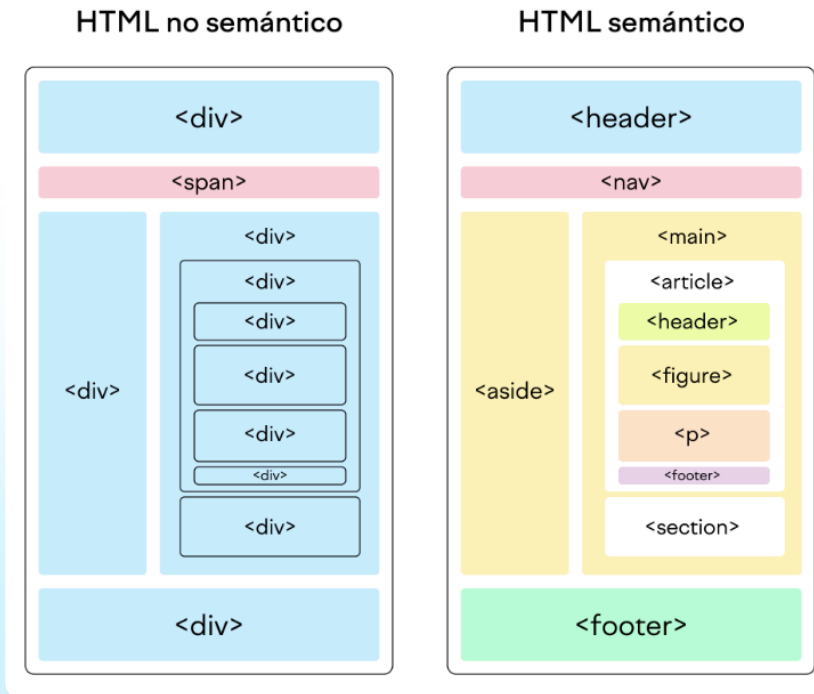
HTML describe **que** es el contenido, no **como** se ve

HTML5 Semántico

Estructura con significado

“ Definición

"HTML semántico es la práctica de usar elementos HTML que describen claramente su significado tanto para el navegador como para el desarrollador — por ejemplo, header, nav, main y article en lugar de div genéricos."



Beneficio 1: SEO

Search Engine Optimization

“ ¿Qué es SEO?

"SEO es la práctica de optimizar sitios web para que aparezcan de forma más prominente en los resultados de los motores de búsqueda, aumentando su visibilidad y tráfico orgánico (no pagado)."

Factores que impactan el ranking



Contenido de calidad

Original, relevante y alineado con la intención de búsqueda



Core Web Vitals

Velocidad de carga (LCP), interactividad (INP), estabilidad visual (CLS)



Mobile-first

Google indexa primero la versión móvil del sitio



Backlinks

Enlaces desde sitios externos con buena reputación



Accesibilidad

Sitios accesibles mejoran engagement y reducen bounce rate



HTML Semántico

Bots entienden la estructura y jerarquía del contenido

HTML Semántico es Bot-Friendly

¿Por qué los bots prefieren HTML semántico?



Provee un mapa del sitio

Tags como `nav`, `main`, `aside` y `footer` actúan como **señalizaciones** que indican al bot el rol de cada sección.



Reduce la ambigüedad

En vez de depender de **heurísticas o nombres de clases** que varían por sitio, los bots usan elementos estándar con significado universal.



Habilita rich snippets

Tags como `article`, `time` y `figure` alimentan los **resultados enriquecidos** de Google (featured snippets, knowledge panels).



Preparado para IA

Crawlers de IA (Google AI Overviews, ChatGPT) **parsean semántica más fácilmente** que cientos de `div` sin significado.

Otros Beneficios del HTML Semántico

Accesibilidad y mantenibilidad



Accesibilidad

Lectores de pantalla pueden **navegar por landmarks** (header, nav, main, footer) sin depender de clases CSS.

Sitios accesibles tienen mejor **engagement**, menor bounce rate y sesiones más largas — señales que Google sí considera.

</> Mantenibilidad

El código es **autoexplicativo**: `nav` comunica intención mejor que `div class="nav"`.

La estructura semántica permite una **colaboración más fluida** entre equipos de desarrollo, diseño y contenido.



Un sitio semántico bien estructurado es más **accesible**, mejor **indexado** y más fácil de **mantener**.



Tendremos clases dedicadas a **SEO** y **Accesibilidad** más adelante en el curso.

Tags Semánticos Estructurales

Los elementos que dan significado a la estructura de una página

<header>

Encabezado de la página o sección. Contiene logo, navegación, título.

<nav>

Bloque de navegación principal. Links a secciones o páginas del sitio.

<main>

Contenido principal y único de la página. Solo uno por documento.

<section>

Agrupación temática de contenido. Cada sección debería tener un heading.

<article>

Contenido independiente y autocontenido: post de blog, noticia, comentario.

<aside>

Contenido tangencial: sidebar, citas relacionadas, publicidad.

<footer>

Pie de página o sección. Info de autor, copyright, links secundarios.

<figure>

Imagen, diagrama o código con su `<figcaption>` descriptivo.

<time>

Fecha u hora legible por máquinas. Atributo `datetime` para bots y SEO.

<details> & <summary>

Widget de disclosure nativo — sin JavaScript

```
<details>
  <summary>¿Que es HTML semantico?</summary>
  <p>HTML semantico usa tags con significado como header,
</details>
```

ⓘ Haz clic en el triangulo para expandir

👁 Resultado interactivo

▶ ¿Que es HTML semantico?

<dialog> & popover

Modals y popovers nativos del navegador

```
◀!— Dialog modal —▶  
<button  
  onclick="  
    document.getElementById('mi-dialog')  
      .showModal()"  
>  
  Abrir Dialog  
</button>  
<dialog id="mi-dialog">  
  <h3>Dialog nativo</h3>  
  <p>Se cierra con Esc</p>  
  <button  
    onclick="  
      this.closest('dialog').close()"  
  >  
    Cerrar  
</button>
```

Dialog

Abrir Dialog

Popover

Abrir Popover

contenteditable

Editar contenido directamente en el navegador

```
<div  
  contenteditable="true"  
  _ ..
```

Resultado interactivo

Haz clic aquí para editar este texto. Puedes escribir lo que quieras directamente en el navegador.

<canvas>

Superficie de dibujo para gráficos en el navegador



Visualización de datos

Charts, dashboards y gráficos interactivos para representar datos.

`Chart.js` · `D3.js` · `Recharts`



Escenas 3D

Modelos 3D, mundos virtuales y experiencias inmersivas usando WebGL.

`Three.js` · `Babylon.js` · `Spline`



bruno-simon.com (3D portfolio)



threejs.org/examples (WebGL)



WebGL (Web Graphics Library) es la API que permite a `canvas` renderizar gráficos 3D de alto rendimiento usando la GPU.

Input: type="color"

Selector de color nativo del navegador

Resultado interactivo

Elige un color:



Inputs de fecha y rango

Selectores nativos sin librerías externas

Resultado interactivo

date: 14-03-2026 

month: March 2026 

week: Week 11, 2026 

range: 

Input: type="file" & <datalist>

Carga de archivos y autocompletado nativo

Resultado interactivo

Subir imagen:

Choose Files

No file chosen

Buscar lenguaje:

Buscar...

Usar inputs nativos mejora la **UX en dispositivos móviles** con teclados especializados

Parte 2: CSS

La presentacion visual y el diseno de la web

¿Que es CSS?

Cascading Style Sheets — define **como se ve** cada elemento

```
h1 {  
  color: green;  
  font-size: 2rem;  
}  
  
.card {  
  background: #f0f9ff;  
  border: 2px solid #7dd3fc;  
  border-radius: 12px;  
}
```

Selector: a que elemento se aplica

Propiedad: que aspecto cambiar

Valor: como cambiarlo

Resultado visual

Titulo

Este es el contenido de una tarjeta estilizada con CSS. Los bordes, colores y espaciado se definen con reglas.

CSS no es programacion

HTML define el **que**, CSS define el **como se ve**

Selectores CSS

Como seleccionar elementos para aplicar estilos

```
/* Basicos */
h1 { }           /* elemento */
.card { }        /* clase */
#hero { }        /* id */

/* Combinadores */
.card p { }      /* descendiente */
.card > p { }    /* hijo directo */
h2 + p { }       /* hermano adyacente */

/* Pseudo-clases */
a:hover { }      /* estado hover */
li:first-child { } /* primer hijo */
input:focus { }  /* con foco */
tr:nth-child(2n) { } /* pares */

/* Pseudo-elementos */
p::first-line { } /* primera linea */
```

Especificidad

0-0-1 elemento (h1, p, div)

0-1-0 .clase, :hover, [attr]

1-0-0 #id

Casos de uso avanzado

:has() — seleccionar padre segun hijo

:is() / :where() — agrupar selectores

:not() — excluir elementos

Especificidad y la Cascada CSS

Cuando multiples reglas aplican al mismo elemento, ¿cual gana?

“ La Cascada

"CSS aplica estilos en orden de prioridad: origen, especificidad y orden de aparicion. Si dos reglas aplican al mismo elemento, la de mayor especificidad gana."

📊 Calcular especificidad

Cada selector suma puntos en 3 columnas:

A **#id** — vale 1-0-0 (100 puntos)

B **.clase, [attr], :pseudo** — vale 0-1-0

C **elemento, ::pseudo** — vale 0-0-1

$\frac{1}{2}$ Ejemplos de calculo

p

0-0-1

.card p

0-1-1

#hero .title

1-1-0

div#main .card p

1-1-2



!important sobreescribe todo, pero **evitalo** — hace el CSS dificil de mantener.

Ejercicio: ¿Que color tiene el texto?

Analiza la especificidad para predecir el resultado

```
/* Regla 1: especificidad ? */  
p { color: blue; }  
  
/* Regla 2: especificidad ? */  
.content p { color: green; }
```

Solucion

p

0-0-1 blue

.content p

0-1-1 green

#main p

1-0-1 red

.content .highlight

0-2-0 orange

Ganador: #main p

El texto es **rojo** porque #main p tiene especificidad 1-0-1, mayor que 0-2-0 de .content .highlight . La columna A (id) siempre pesa mas que la B (clases).

Features modernas de CSS

Variables, nesting y selectores avanzados

Variables CSS (Custom Properties)

Valores reutilizables en todo el documento

Declarar variables

```
:root {  
  --main-bg: #1a1a2e;  
  --accent: #e94560;  
  --text: #eee;  
}
```

:root define variables globales

Usar variables

```
.card {  
  background: var(--main-bg);  
  color: var(--text);  
  border: 1px solid var(--accent);  
}
```

var() accede al valor de la variable



Variables CSS permiten **temas dinamicos** y **consistencia visual** en toda la aplicacion

CSS Nesting

Estilos anidados directamente en CSS nativo

Sin nesting (tradicional)

```
.card {  
  padding: 1rem;  
}  
  
.card h2 {  
  color: blue;  
}  
  
.card p {  
  opacity: 0.7;  
}
```

Con nesting (moderno)

```
.card {  
  padding: 1rem;  
  
  h2 {  
    color: blue;  
  }  
  
  p {  
    opacity: 0.7;  
  }  
}
```



Nesting nativo en CSS: **mas legible, modular y mantenible** (inspirado en Sass)

Layout moderno: Flexbox y Grid

Las dos herramientas fundamentales para distribuir elementos

CSS Flexbox

Distribucion eficiente en una dimension

```
.navbar {  
  display: flex;
```

`justify-content`

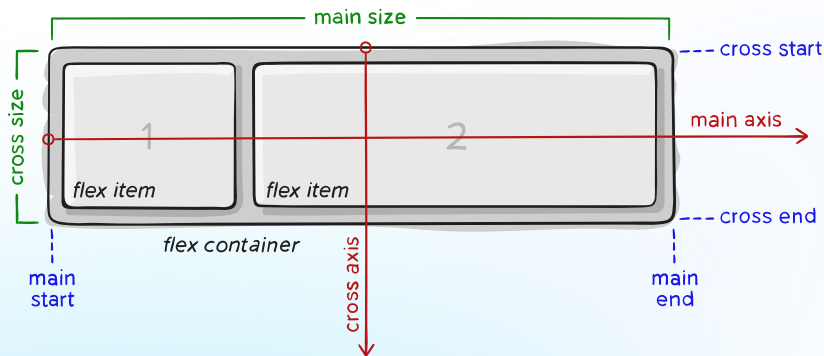
`align-items`

`flex-direction`

`gap`

[Flexbox Guide \(CSS Tricks\)](#)

[Guide \(Josh W. Comeau\)](#)



CSS Grid

Layout bidimensional con filas y columnas

`display: grid`

Activa Grid en el contenedor padre.

`grid-template-columns`

Define cuántas columnas y su ancho. Ej: `200px 1fr` , `repeat(3, 1fr)`

`grid-template-rows`

Define las filas y su alto. Ej: `auto 1fr auto`

`grid-template-areas`

Nombra zonas del grid para asignar elementos con `grid-area` .

`gap`

Espacio entre filas y columnas. Ej: `12px` , `1rem 2rem`

`grid-area`

Asigna un hijo a una zona nombrada. Ej: `grid-area: header`

💡 **fr** = fracción del espacio libre. En `200px 1fr` , el navegador da 200px a la primera columna y **todo el resto** a la segunda.

[🔗 Grid Guide \(CSS Tricks\)](#)

[🔗 Guide \(Josh W. Comeau\)](#)

CSS Grid: Ejemplo interactivo

Layout con grid-template-areas — edita el código en vivo

[↗ Abrir MDN Playground — CSS Grid](#)

Flexbox vs Grid: ¿Cual usar?

Depende de la dimension del layout

→ Flexbox: 1 dimension

Ideal cuando los elementos se muestran **en linea** (horizontal o vertical).

- ✓ Barras de navegacion
- ✓ Listas de items
- ✓ Alineacion de componentes

Grid: 2 dimensiones

Ideal cuando necesitas controlar **filas y columnas** simultaneamente.

- ✓ Layouts de pagina completa
- ✓ Dashboards y galerias
- ✓ Grillas de tarjetas

💡 **Flexbox para componentes, Grid para layout** – pero no hay regla absoluta

Parte 3: Herramientas CSS

Frameworks y preprocesadores para desarrollo eficiente

Ecosistema de herramientas CSS

Preprocesadores y frameworks populares



Sass

Extension de CSS con variables, nesting, mixins y funciones. El preprocesador mas maduro y popular.



Bootstrap

Framework front-end con componentes y estilos listos para usar. Muy popular para prototipos rapidos.



Tailwind CSS

Framework utility-first con clases predefinidas. **El enfoque moderno que profundizaremos hoy.**

[🔗 tailwindcss.com](https://tailwindcss.com)



El framework utility-first que cambio el desarrollo web

¿Que es Tailwind CSS?

El framework CSS mas usado — y el preferido por herramientas de AI

“ Concepto

"Un framework de CSS con clases de utilidad predefinidas como flex, pt-4, text-center y rotate-90 que se componen directamente en el HTML para construir cualquier diseno."

¿Por que AI prefiere Tailwind?

- ✓ **Todo en un archivo** — AI no necesita razonar entre HTML y CSS separados
- ✓ **Naming predecible** — escalas consistentes que los LLMs reproducen bien
- ✓ **Sin cascada oculta** — cada elemento tiene todo su estilo visible

Sistema de diseno integrado

- ✓ **Escala de colores** — paleta de 22 colores x 11 tonos predefinidos
- ✓ **Spacing consistente** — escala de 4px (p-1=4px, p-2=8px...)
- ✓ **Tipografia** — text-xs a text-9xl con line-height incluido



v0, Lovable, Bolt.new y Claude generan **Tailwind por defecto** — aprender Tailwind es aprender a trabajar con AI

Clases utilitarias de Tailwind

Cada clase hace exactamente una cosa



Spacing

`p-4 m-2 px-6 py-3 mt-8 gap-4`

Aa Tipografia

`text-lg font-bold text-center italic`



Colores

`bg-blue-500 text-white border-gray-200`

Layout

`flex grid items-center justify-between`



Dimensiones

`w-full h-screen max-w-lg min-h-0`



Responsive

`sm:flex md:grid-cols-2 lg:text-xl`



¿Por que utility-first?

- ✓ No inventas nombres de clases
- ✓ CSS no crece con el proyecto
- ✓ Cambios locales y seguros

Ejemplo: tarjeta de producto

Construida solo con clases utilitarias

```
<div class="group relative max-w-sm rounded-lg p-3 shadow">
  
  <div class="mt-4 flex justify-between">
    <div>
      <h3 class="text-sm text-gray-700">
        <a href="#">
          <span aria-hidden="true" class="absolute inset-0 sr-only">
            Basic Tee
          </span>
        </a>
      </h3>
      <p class="mt-1 text-sm text-gray-500">Black</p>
    </div>
    <p class="text-sm font-medium text-gray-900">$35</p>
  </div>
</div>
```

Ver en Tailwind Play

💡 Iterar el diseño es cambiar clases — sin tocar archivos CSS separados

Trade-offs de Tailwind CSS

Ventajas y desventajas a considerar

✓ Ventajas

- ⚡ **Iteracion rapida** — cambiar estilos es editar clases
- 📏 **Sistema de diseno integrado** — valores consistentes
- 🗑️ **CSS minimo** — purge elimina clases no usadas
- 🤖 **AI-friendly** — generadores de codigo lo prefieren
- 🗑️ **Sin naming wars** — no inventas nombres de clases

⚠️ Desventajas

- </> **HTML verboso** — muchas clases por elemento
- 🧠 **Curva de aprendizaje** — hay que memorizar utilidades
- 📁 **Separacion de concerns** — estilos mezclados con HTML
- 📄 **Repeticion** — sin componentes, se duplican clases

💡 La solucion a la repeticion: 1. **extraer componentes** 2. crear clases CSS con `@apply`

Tailwind simplifica Grid

¿Qué significan las clases de Tailwind para Grid?

grid-cols-3

= `grid-template-columns: repeat(3, 1fr)`

3 columnas de igual ancho. 1fr = 1 fracción del espacio disponible.

grid-rows-2

= `grid-template-rows: repeat(2, minmax(0, 1fr))`

2 filas de igual alto, cada una ocupando 1 fracción.

grid-cols-[200px_1fr]

= `grid-template-columns: 200px 1fr`

Primera columna fija de 200px, la segunda ocupa el resto.

grid-rows-[auto_1fr_auto]

= `grid-template-rows: auto 1fr auto`

Header y footer ajustan a su contenido, el centro ocupa el resto.

gap-6

= `gap: 1.5rem (24px)`

Espacio entre filas y columnas. También: `gap-x-4` y `gap-y-2` por separado.

col-span-2

= `grid-column: span 2 / span 2`

El elemento ocupa 2 columnas. Útil para headers o footers anchos.

💡 **fr** = fracción del espacio libre. En `200px 1fr`, el navegador da 200px a la primera columna y **todo el resto** a la segunda.

Tailwind: Responsive Design

Breakpoints como prefijos de clase

Mobile-first por defecto

```
<div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4">
```



Base

1 columna



sm: 640px

2 columnas



md: 768px

3 columnas



lg: 1024px

4 columnas



Estados interactivos

```
hover:bg-blue-600  
focus:ring-2  
active:scale-95  
dark:bg-gray-800
```



Dark Mode

```
bg-white dark:bg-gray-900  
text-black dark:text-white
```

Prefijo `dark:` para estilos oscuros



Valores arbitrarios

```
w-[350px] bg-[#1a1a2e]  
grid-cols-[200px_1fr]
```

Corchetes para valores personalizados

¿Que aprendimos hoy?

Resumen de la clase



HTML: estructura de la web

Lenguaje de marcado que define el significado y estructura del contenido



CSS: presentacion visual

Lenguaje de estilo para definir como se ve cada elemento, con variables y nesting



Flexbox y Grid

Layout unidimensional (Flexbox) y bidimensional (Grid) para distribuir elementos



Tailwind CSS

Framework utility-first con clases predefinidas, responsive y estados interactivos

Recursos recomendados

Para profundizar en los temas de hoy



MDN Web Docs

Referencia oficial de HTML y CSS



CSS Tricks – Grid Guide

Guia completa e interactiva de Grid



Josh W. Comeau

Guías interactivas de Flexbox y Grid



CSS Tricks – Flexbox Guide

Guia completa e interactiva de Flexbox



Tailwind CSS Docs

Documentacion oficial de Tailwind



CSSBattle

Sitio para practicar CSS replicando imagenes

¿Preguntas?

Clase 3 – Fundamentos del Web Moderno



PONTIFICIA
UNIVERSIDAD
CATÓLICA
DE CHILE

IIC2182 – Primer Semestre 2026